DATA MODELLING – EXAMPLE - TOI (ENSEMBLE)

PURPOSE: This notebook utilises TESS Projects Candidate data to detect short-period planets which may include sub-neptune candidates.

Machine learning (ML) models are applied to cleaned, balanced data: Support Vector Classifier (SVC), Random Forest (RF), Light Gradient Boosting Machine (LGBM) and Multi-Layer Perceptron (MLP).

The Ensemble Learning (EL) model is constructed using the above optimal machine learning algorithms, each utilising varying feature sets.

PRE-REQUISITE PROCESSING:
TOI MODELLING

INPUT:
Cleaned TOI data sample as at 30 March 2025.

Optimised Machine Learning models trained with varying feature sets for LightGBM, Random Forest (RF), Support Vector (SVC), Multi-Layer Perceptron (MLP).

OUTPUT:
Constructed Ensemble Model composed of selected optimum models.

PROCESSING:
Determines which optimised machine learning algortihms to use for the Ensemble Learning model construction.

```
from google.colab import drive
drive.mount('/content/drive')
```

```
Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).
```

DECLARATIONS

```
# PYTHON LIBRARIES
import warnings # Display warning messages

warnings.filterwarnings('ignore')

import calendar       # Convert to names of month
import csv            # Reading in CSV files
import numpy as np    # Operations on arrays
import pandas as pd   # Data manipulation and analysis
import datetime as dt # Date and time manipulation
import math           # Mathematical functions
import seaborn as sns # Seaborn library
import os             # Directory

import matplotlib.pyplot as plt  # Plots
import matplotlib.dates as mdate # Date manipulation for graphical display

from math import sqrt # SQRT

from scipy.stats import spearmanr # Spearman Correlation

import statsmodels.api as sm # Statsmodels library

# SKLEARN Library
from sklearn import datasets
from sklearn.model_selection import cross_val_predict              # ANN - Multilayer Perceptron model
from sklearn.model_selection import train_test_split               # Split method
from sklearn.model_selection import GridSearchCV                   # Best parameter value
from sklearn.model_selection import cross_val_score                # Cross validation
from sklearn.metrics import classification_report, confusion_matrix # Reporting Metrics
from sklearn.metrics import accuracy_score                         # Reporting Metrics

from sklearn.metrics import mean_squared_error                     # RMSE
from sklearn.neural_network import MLPClassifier                   # ANN Model
from sklearn.preprocessing import StandardScaler                   # Standardises features by removing the mean and scaling to unit varia
from sklearn.pipeline import Pipeline                              # Pipeline Library


# Magic command to plot figures in the Notebook
%matplotlib inline

from IPython import get_ipython
get_ipython().magic('reset -sf')
```

```
# INSTALL PYSPARK

!pip install pyspark
```

```
Requirement already satisfied: pyspark in /usr/local/lib/python3.12/dist-packages (4.0.2)
Requirement already satisfied: py4j<0.10.9.10,>=0.10.9.7 in /usr/local/lib/python3.12/dist-packages (from pyspark) (0.10.9.9)
```

```
# INSTALL IMBALANCED-LEARN (SMOTE)
!pip install imbalanced-learn
```

```
Requirement already satisfied: imbalanced-learn in /usr/local/lib/python3.12/dist-packages (0.14.1)
Requirement already satisfied: numpy<3,>=1.25.2 in /usr/local/lib/python3.12/dist-packages (from imbalanced-learn) (2.0.2)
Requirement already satisfied: scipy<2,>=1.11.4 in /usr/local/lib/python3.12/dist-packages (from imbalanced-learn) (1.16.3)
Requirement already satisfied: scikit-learn<2,>=1.4.2 in /usr/local/lib/python3.12/dist-packages (from imbalanced-learn) (1.6.1)
Requirement already satisfied: sklearn-compat<0.2,>=0.1.5 in /usr/local/lib/python3.12/dist-packages (from imbalanced-learn) (0.1.5)
Requirement already satisfied: joblib<2,>=1.2.0 in /usr/local/lib/python3.12/dist-packages (from imbalanced-learn) (1.5.3)
Requirement already satisfied: threadpoolctl<4,>=2.0.0 in /usr/local/lib/python3.12/dist-packages (from imbalanced-learn) (3.6.0)
```

```
# INSTALL SHAP

!pip install shap
```

```
Requirement already satisfied: shap in /usr/local/lib/python3.12/dist-packages (0.51.0)
Requirement already satisfied: numpy>=2 in /usr/local/lib/python3.12/dist-packages (from shap) (2.0.2)
Requirement already satisfied: scipy in /usr/local/lib/python3.12/dist-packages (from shap) (1.16.3)
Requirement already satisfied: scikit-learn in /usr/local/lib/python3.12/dist-packages (from shap) (1.6.1)
Requirement already satisfied: pandas in /usr/local/lib/python3.12/dist-packages (from shap) (2.2.2)
Requirement already satisfied: tqdm>=4.27.0 in /usr/local/lib/python3.12/dist-packages (from shap) (4.67.3)
Requirement already satisfied: packaging>20.9 in /usr/local/lib/python3.12/dist-packages (from shap) (26.1)
Requirement already satisfied: slicer==0.0.8 in /usr/local/lib/python3.12/dist-packages (from shap) (0.0.8)
Requirement already satisfied: numba in /usr/local/lib/python3.12/dist-packages (from shap) (0.60.0)
Requirement already satisfied: llvmlite in /usr/local/lib/python3.12/dist-packages (from shap) (0.43.0)
Requirement already satisfied: cloudpickle in /usr/local/lib/python3.12/dist-packages (from shap) (3.1.2)
Requirement already satisfied: typing-extensions in /usr/local/lib/python3.12/dist-packages (from shap) (4.15.0)
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.12/dist-packages (from pandas->shap) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.12/dist-packages (from pandas->shap) (2025.2)
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.12/dist-packages (from pandas->shap) (2026.1)
Requirement already satisfied: joblib>=1.2.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn->shap) (1.5.3)
Requirement already satisfied: threadpoolctl>=3.1.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn->shap) (3.6.0)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateutil>=2.8.2->pandas->shap) (1.17.0)
```

Double-click (or enter) to edit

DECLARATIONS

```
# REPORTING METRICS
from sklearn.metrics import confusion_matrix,  ConfusionMatrixDisplay, balanced_accuracy_score, classification_report, cohen_kappa_score,
from sklearn.metrics import accuracy_score

from sklearn.metrics import roc_curve, auc
from sklearn.metrics import mean_squared_error,r2_score             # RMSE, Chi-square
```

```
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import GridSearchCV
from sklearn.model_selection import train_test_split, RandomizedSearchCV
```

```
from scipy.stats import randint
```

```
import matplotlib.pyplot as plt
```

OPTIMISED MODELS - HYPERPARAMETER TUNING

```
# SKLEARN Library
from sklearn import datasets
from sklearn.model_selection import cross_val_predict               # ANN - Multilayer Perceptron model
from sklearn.model_selection import train_test_split                # Split method
from sklearn.model_selection import GridSearchCV                    # Best parameter value
from sklearn.model_selection import RepeatedKFold

from sklearn.neural_network import MLPClassifier                    # ANN Model
from sklearn.preprocessing import StandardScaler                    # Standardises features by removing the mean and scaling to unit varia

from sklearn.svm import SVC                                         # Support Vector Classifier
from sklearn.ensemble import RandomForestClassifier                 # Random Forest
from lightgbm import LGBMClassifier                                 # Light Gradient Boosting Machine
from sklearn.neural_network import MLPClassifier                    # Multi-Layer Perceptron
```

```
# CLASSIFICATION REPORT
from sklearn.metrics import classification_report, confusion_matrix # Reporting Metrics
from sklearn.metrics import accuracy_score                          # Reporting Metrics
```

```
# PIPE LINE LIBRARY
from sklearn.pipeline import make_pipeline # library to make a pipeline
from sklearn.pipeline import Pipeline
```

```
# REPORTING METRICS

from sklearn.model_selection import cross_val_score                 # Cross validation
from sklearn.metrics import accuracy_score                          # Reporting Metrics
```

```
from sklearn.metrics import confusion_matrix,  ConfusionMatrixDisplay, balanced_accuracy_score, classification_report, cohen_kappa_score,
from sklearn.metrics import accuracy_score

from sklearn.metrics import roc_curve, auc
from sklearn.metrics import mean_squared_error,r2_score            # RMSE, Chi-square
```

1.READ REQUIRED FILES

1.1 - READ PRE-REQUISITE DATA

Read input data which has been cleaned, split, scaled and class balanced (using SMOTE) into train and test subsets of varying feature lengths for:

- Feature matrix (X)
- Target variable (y)

1.2 - ASSIGN SELECTED FEATURES

Assign feature subset of preferred length for modelling purposes.

```
selected_features = X_train_resampled_preferred.columns
selected_features
```

```
Index(['log_pl_trandep', 'log_pl_eqt', 'ratio_pl_eqt', 'log_st_dist',
       'log_ratio_rade_eqt', 'st_mass_est', 'ratio_pl_orbper',
       'ratio_eqt_teff', 'log_pl_insol', 'log_pl_rade', 'log_pl_orbper',
       'log_Predicted Mass (M_Earth)', 'log_st_rad', 'log_st_teff',
       'log_pl_trandurh', 'st_logg', 'log_ratio_pl_st_rad', 'st_tmag'],
      dtype='object')
```

Double-click (or enter) to edit

2.READ MODELS

2.1 - LOAD OPTIMISED MODELS

```
import joblib

# LOAD PREFERRED OPTIMISED MODELS

model_LGBM_opt_loaded = joblib.load('/content/drive/MyDrive/model_LGBM_opt_PORT.joblib')
model_SVC_opt_loaded = joblib.load('/content/drive/MyDrive/model_SVC_opt_PORT.joblib')
```

```
import joblib

model_MLP_opt_preferred_loaded = joblib.load('/content/drive/MyDrive/model_MLP_opt_preferred_PORT.joblib')
model_RF_opt_preferred_loaded = joblib.load('/content/drive/MyDrive/model_RF_opt_preferred_PORT.joblib')
```

```
model_SVC_opt_loaded, 'full'),
        (model_RF_opt_preferred_loaded,'preferred'),
        (model_LGBM_opt_loaded, 'full'),
        (model_MLP_opt_preferred_loaded, 'preferred')
```

2.2 - CALCULATE PER-MODEL PER-MODEL PROBABILITY

```
y_prob_SVC = model_SVC_opt_loaded.predict_proba(X_test_scaled)[:, 1]
y_prob_LGBM = model_LGBM_opt_loaded.predict_proba(X_test_scaled)[:, 1]
```

```
y_prob_RF_preferred = model_RF_opt_preferred_loaded.predict_proba(X_test_scaled_preferred)[:, 1]
y_prob_MLP_preferred = model_MLP_opt_preferred_loaded.predict_proba(X_test_scaled_preferred)[:, 1]
```

3- GENERATE HYBRID ENSEMBLE MODEL - SOFT VOTING WRAPPER

The ensemble model was obtained via soft voting by averaging the class posterior probabilities produced by the RF, SVC, LightGBM, and MLP classifiers.

Each base learner was trained on a model-specific feature representation, with preprocessing steps encapsulated within individual pipelines.

The soft voting ensemble wrapper can cater for ML models being trained using various feature lengths. It produces the original line: "np.mean([y_prob_SVC, y_prob_RF_preferred, y_prob_LGBM, y_prob_MLP_preferred])" dynamically for any dataset.

Default threshold set to 0.5.

NOTE: Preprocessors (for prediction) are already embedded inside each pipeline used during ML creation.

3.1 - CREATE WRAPPER FUNCTION FOR ENSEMBLE MODEL

```
# FUNCTION TO CREATE ENSEMBLE MODEL

import numpy as np
import pandas as pd

class SoftVotingEnsemble:
    def __init__(self, pipelines_info, preferred_feature_names):
        """
        pipelines_info: list of tuples (fitted_pipeline, 'full' or 'preferred')
        preferred_feature_names: list of column names for preferred features
        """
        self.pipelines_info = pipelines_info
        self.preferred_feature_names = preferred_feature_names

    def predict_proba(self, X_df_scaled): # EXPECTS A SCALED DATAFRAME WITH FULL FEATURE SET
        probs = []
        for pipe, feature_type in self.pipelines_info:
            if feature_type == 'full':
                # For full feature pipelines, use X_df_scaled directly (assumed to be the full feature set)
                X_subset = X_df_scaled
            elif feature_type == 'preferred':
                # For preferred feature pipelines, select columns from X_df_scaled
                X_subset = X_df_scaled[self.preferred_feature_names]
            else:
                raise ValueError(f"Unknown feature_type: {feature_type}")
            probs.append(pipe.predict_proba(X_subset)[:, 1])
        return np.mean(probs, axis=0)

    def predict(self, X_df_scaled, threshold=0.5):
        return (self.predict_proba(X_df_scaled) >= threshold).astype(int)
```

3.2 - INSTANTIATE THE ENSEMBLE MODEL USING TRAINED PIPELINES

```
pca=PCA()

# RANDOM FOREST
predictor = RandomForestClassifier(random_state=42)
pipe_RF_hyper = make_pipeline(pca, predictor)

# SUPPORT VECTOR CLASSIFIER
predictor = SVC(random_state=0, probability =  True)
pipe_SVC_hyper = make_pipeline(pca, predictor)

# MULTI-LAYER PERCEPTRON
predictor = MLPClassifier(random_state=42, max_iter=1000)
pipe_MLP_hyper = make_pipeline(pca, predictor)

# LIGHT GRADIENT BOOSTING
predictor = LGBMClassifier(random_state=42)
pipe_LGBM_hyper = make_pipeline(pca, predictor)
```

```
model_ensemble = SoftVotingEnsemble(
    pipelines_info=[
        (model_SVC_opt_loaded, 'full'),
        (model_RF_opt_preferred_loaded, 'preferred'),
        (model_LGBM_opt_loaded, 'full'),
        (model_MLP_opt_preferred_loaded, 'preferred')
    ],
    preferred_feature_names=selected_features # SELECTED FEATURES PREVIOUSLY DEFINED
)
```

3.3 - EVALUATE PERFORMANCE METRICS

```
# DETERMINE PROBAILITIES ACROSS ALL THRESHOLDS

y_prob_ensemble_2 = model_ensemble.predict_proba(X_test_scaled)
```

```
# THRESHOLD FOR FINAL PREDICTION - APPLY A DECISION THRESHOLD (e.g 0.5)

y_pred_ensemble_thresh_default_2 = (y_prob_ensemble_2 > 0.5).astype(int)
```

```
from sklearn.metrics import classification_report, roc_auc_score, confusion_matrix

print(classification_report(y_test, y_pred_ensemble_thresh_default_2))
print("Unweighted Ensemble ROC AUC:", roc_auc_score(y_test, y_prob_ensemble_2))
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred_ensemble_thresh_default_2))
```

```
              precision    recall  f1-score   support

           0       0.90      0.86      0.88       218
           1       0.85      0.89      0.87       193

    accuracy                           0.87       411
   macro avg       0.87      0.87      0.87       411
weighted avg       0.87      0.87      0.87       411

Unweighted Ensemble ROC AUC: 0.9468555402386272
Confusion Matrix:
 [[187  31]
 [ 21 172]]
```

```
# EVALUATE THE MODEL
report = classification_report(y_test,y_pred_ensemble_thresh_default_2, output_dict=True)

# CONVERT THE REPORT TO A DATAFRAME
df_report = pd.DataFrame(report).transpose()


# RENAME THE INDEX LABELS
df_report.rename(index={'0': 'FALSE POSITIVE', '1': 'CONFIRMED'}, inplace=True)

# DISPLAY THE DATAFRAME
print(df_report, '\n')
```

```
               precision    recall  f1-score     support
FALSE POSITIVE   0.899038  0.857798  0.877934  218.000000
CONFIRMED        0.847291  0.891192  0.868687  193.000000
accuracy         0.873479  0.873479  0.873479    0.873479
macro avg        0.873165  0.874495  0.873311  411.000000
weighted avg     0.874738  0.873479  0.873592  411.000000
```

```
# Style the DataFrame
styled_report = df_report.style \
    .background_gradient(cmap='YlGnBu') \
    .set_caption('ENSEMBLE (Hybrid Wrap - Default) - Classification Report') \
    .format({'precision': '{:.2f}', 'recall': '{:.2f}', 'f1-score': '{:.2f}', 'support': '{:.0f}'}) \
    .set_properties(**{'text-align': 'center'}) \
    .set_table_styles([{
        'selector': 'caption',
        'props': 'caption-side: top; font-size: 16px; font-weight: bold;'
    }])
```

```
plt.figure(figsize=(16, 16))
display(styled_report)
```

ENSEMBLE (Hybrid Wrap - Default) - Classification Report

	precision	recall	f1-score	support
FALSE POSITIVE	0.90	0.86	0.88	218
CONFIRMED	0.85	0.89	0.87	193
accuracy	0.87	0.87	0.87	1
macro avg	0.87	0.87	0.87	411
weighted avg	0.87	0.87	0.87	411

```
<Figure size 1600x1600 with 0 Axes>
```

```
# EVALUATE THE MODEL - CONFUSION MATRIX
conf_matrix = confusion_matrix(y_test, y_pred_ensemble_thresh_default_2)
f1 = f1_score(y_test, y_pred_ensemble_thresh_default_2, average='weighted')
# report = classification_report(y_test, y_predict_MLP_opt)
accuracy = balanced_accuracy_score(y_test, y_pred_ensemble_thresh_default_2)
kappa = cohen_kappa_score(y_test, y_pred_ensemble_thresh_default_2)

print(f"F1 Score: {f1}")
print(f"Kappa Score: {kappa}")
print(f"Accuracy Score: {accuracy}")
print(f"Confusion Matrix:\n{conf_matrix}")
print(report)
```

```
F1 Score: 0.8735918175622094
Kappa Score: 0.7467712505035664
Accuracy Score: 0.8744949374910871
Confusion Matrix:
[[187  31]
 [ 21 172]]
{'0': {'precision': 0.8990384615384616, 'recall': 0.8577981651376146, 'f1-score': 0.8779342723004695, 'support': 218.0}, '1': {'precision':
```

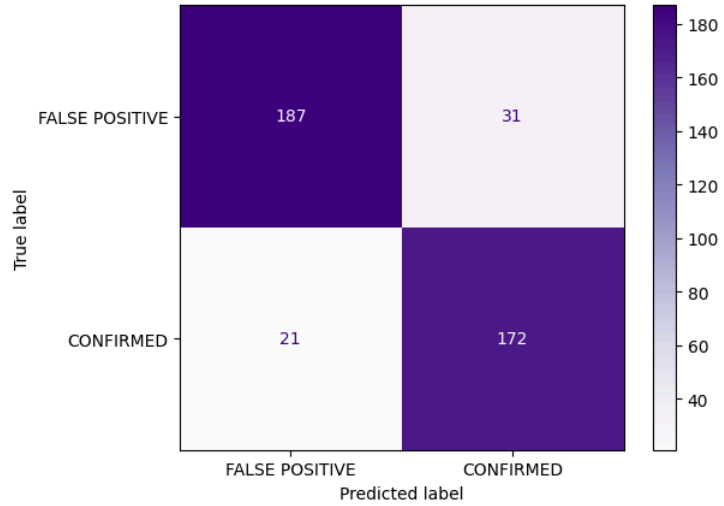
```
# CREATE A CONFUSION MATRIX WITH CUSTOM LABELS

from sklearn.metrics import classification_report, confusion_matrix # Reporting Metrics

cm_ensemble_thresh_default_2 = confusion_matrix(y_test, y_pred_ensemble_thresh_default_2)
```

```
disp = ConfusionMatrixDisplay(confusion_matrix=cm_ensemble_thresh_default_2, display_labels=['FALSE POSITIVE', 'CONFIRMED'])

# PLOT THE CONFUSION MATRIX
disp.plot(cmap='Purples')
plt.show()
```



3.4 - AUC COMPARISON PLOT (ROC OVERLAY)

```
# COMPUTE ROC CURVE AND AREA

# SVC
fpr_SVC_opt, tpr_SVC_opt, _ = roc_curve(y_test, y_prob_SVC)
roc_auc_SVC_opt = auc(fpr_SVC_opt, tpr_SVC_opt)

# RF
fpr_RF_opt_preferred, tpr_RF_opt_preferred, _ = roc_curve(y_test, y_prob_RF_preferred)
roc_auc_RF_opt_preferred = auc(fpr_RF_opt_preferred, tpr_RF_opt_preferred)

# LGBM
fpr_LGBM_opt, tpr_LGBM_opt, _ = roc_curve(y_test, y_prob_LGBM)
roc_auc_LGBM_opt = auc(fpr_LGBM_opt, tpr_LGBM_opt)

# MLP
fpr_MLP_opt_preferred, tpr_MLP_opt_preferred, _ = roc_curve(y_test, y_prob_MLP_preferred)
roc_auc_MLP_opt_preferred = auc(fpr_MLP_opt_preferred, tpr_MLP_opt_preferred)

# HYBRID WRAPPED ENSEMBLE (DEFAULT)
fpr_ensemble_thresh_default_2, tpr_ensemble_thresh_default_2, _ = roc_curve(y_test, y_prob_ensemble_2)
roc_auc_ensemble_thresh_default_2 = auc(fpr_ensemble_thresh_default_2, tpr_ensemble_thresh_default_2)
```

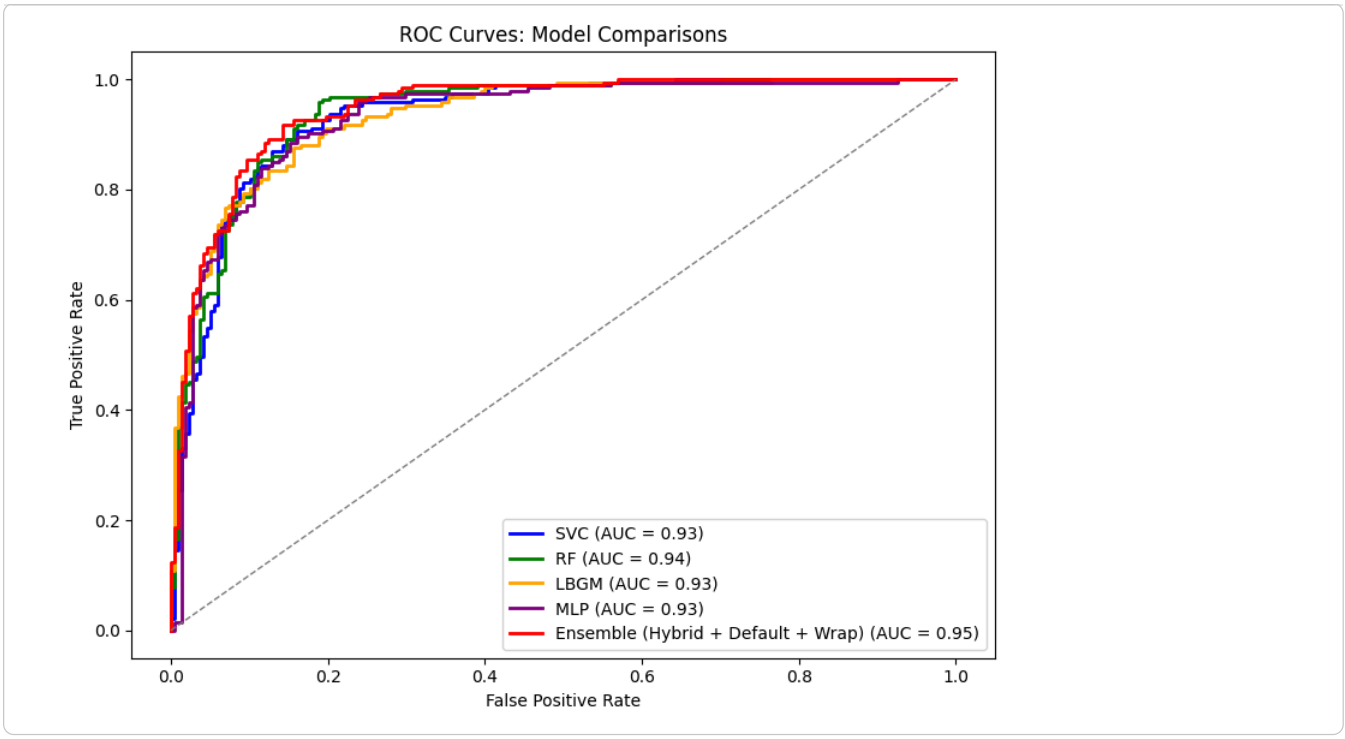
```
from sklearn.metrics import roc_curve, auc
import matplotlib.pyplot as plt

plt.figure(figsize=(8, 6))

plt.plot(fpr_SVC_opt, tpr_SVC_opt, color='blue', lw=2, label=f'SVC (AUC = {roc_auc_SVC_opt:.2f})')
plt.plot(fpr_RF_opt_preferred, tpr_RF_opt_preferred, color='green', lw=2, label=f'RF (AUC = {roc_auc_RF_opt_preferred:.2f})')
plt.plot(fpr_LGBM_opt, tpr_LGBM_opt, color='orange', lw=2, label=f'LBGM (AUC = {roc_auc_LGBM_opt:.2f})')
plt.plot(fpr_MLP_opt_preferred, tpr_MLP_opt_preferred, color='purple', lw=2, label=f'MLP (AUC = {roc_auc_MLP_opt_preferred:.2f})')

# OPTIMUM HYBRID ENSEMBLE
plt.plot(fpr_ensemble_thresh_default_2, tpr_ensemble_thresh_default_2, color='red', lw=2, label=f'Ensemble (Hybrid + Default + Wrap) (AUC

plt.plot([0, 1], [0, 1], color='grey', linestyle='--', lw=1)
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('ROC Curves: Model Comparisons')
plt.legend()
plt.tight_layout()
plt.show()
```


ROC Curves: Model Comparisons
True Positive Rate
False Positive Rate
0.0
0.2
0.4
0.6
0.8
1.0
SVC (AUC = 0.93)
RF (AUC = 0.94)
LBGM (AUC = 0.93)
MLP (AUC = 0.93)
Ensemble (Hybrid + Default + Wrap) (AUC = 0.95)